

Absolutely. You now have enough pieces to build the system properly from the beginning.

One important correction before the full guide: **Instagram image publishing is fully tested and working. Facebook Page text publishing is tested and working. Facebook Page image publishing still needs one final image test.** I do not want to claim that part is complete until you actually see the photo on the Page.

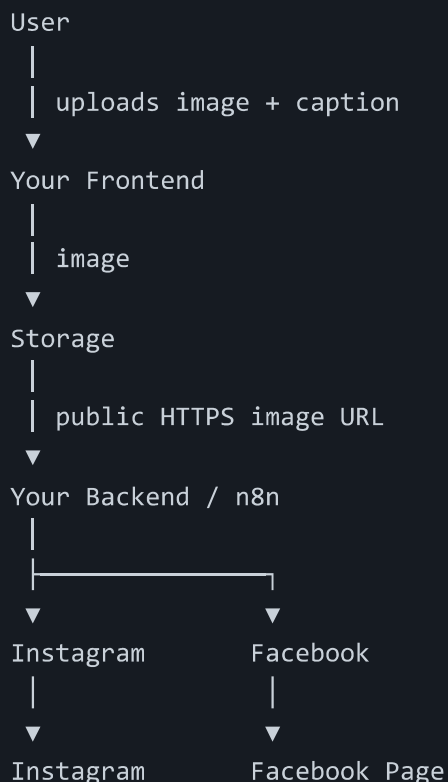
Also, the Meta developer UI and API versions change over time. I could not retrieve usable current official Meta documentation from web search just now, so the API paths below are based on the exact requests that you successfully used in your setup. For your first n8n build, use the same versions that already worked.

1. First understand the ABCD

Think of the whole project as four layers.

```
A = Your user
B = Your backend / n8n
C = Meta API
D = Facebook / Instagram
```

Your actual application will eventually look like:



For example:

```
User selects:  
photo.jpg  
Caption: "Hello world"  
Platform: Both
```

Your backend turns the image into something like:

```
https://your-storage.com/images/abc123.jpg
```

Then n8n uses that URL.

2. Very important: local image path vs public image URL

This is one of the most important concepts.

Your computer might have:

```
C:\Users\Yuvaraj\Pictures\photo.jpg
```

Meta cannot use that.

Meta needs something like:

```
https://your-storage.com/photo.jpg
```

because Meta's servers need to download the image.

So:

```
Local file  
  ↓  
Upload  
  ↓  
Cloud/object storage  
  ↓  
Public HTTPS URL  
  ↓  
Instagram/Facebook
```

For your first API test you used an Unsplash URL, which is why it worked.

Later your application will replace that with your own uploaded-image URL.

3. META APP ARCHITECTURE

You created one Meta app:

```
VelorQ
```

Inside that one app you configured two separate integrations:

```
VelorQ
├── Instagram API with Instagram Login
└── Facebook Login for Business
    └── Facebook Page
```

That is the correct architecture.

Do **not** create separate Meta apps just because you have Instagram and Facebook.

4. INSTAGRAM — COMPLETE SETUP FROM ZERO

Step 4.1 — Instagram account

Create/use an Instagram account.

Change it to:

```
Professional account
```

The account you're using is:

```
@velar.q
```

You already did this.

5. Instagram Meta Developer setup

Open:

```
https://developers.facebook.com/
```

Then:

```
My Apps  
→ VelorQ  
→ Use cases
```

You selected:

```
Manage messaging & content on Instagram
```

Open:

```
Customize
```

Then use:

```
API setup with Instagram Login
```

6. Instagram permissions

Inside:

```
Instagram API  
→ API setup with Instagram Login
```

You added the required permissions.

Most importantly for your application you verified:

```
instagram_content_publish
```

and Meta showed:

```
Ready for testing
```

This permission is the one that matters for your image-posting goal.

7. Instagram Tester

Go to:

```
App roles
```

Add your Instagram account as:

```
Instagram Tester
```

For you:

```
@velar.q
```

Then log into that Instagram account and accept the invitation.

You already completed this.

8. Instagram account connection

Go to:

```
Instagram API  
→ API setup with Instagram Login  
→ Generate access tokens
```

Click:

Add account

Select your Instagram account.

You did this.

9. Instagram access token

Now:

Generate access token

Meta generates an access token for the connected Instagram account.

Conceptually:

```
Meta App
  ↓
Instagram authorization
  ↓
Instagram Account
  ↓
Instagram Access Token
```

The token should be treated as a secret.

Never put it in:

```
Frontend JavaScript
GitHub
public screenshots
chat messages
source code
```

Store it in your backend/n8n credentials.

10. Instagram account ID

Meta showed you the numeric Instagram ID.

Do not use:

```
@velar.q
```

for the API object ID.

Use:

```
YOUR_IG_USER_ID
```

In your successful test it was a numeric ID.

This difference was the reason you initially got:

```
Unsupported post request
```

because you used:

```
/velar.q/media
```

instead of:

```
/YOUR_IG_USER_ID/media
```

11. Instagram token validation

Meta's API Integration Helper lets you enter the token:

```
Enter your access token  
→ Validate
```

You verified that it works.

The helper also showed the account.

The `Hello World` section you saw was the **messaging tester**.

That is not the image publishing flow.

So don't get confused when you see:

```
Send messages
Text
Hello World
```

That is for messaging.

12. Instagram actual publishing API

This is the most important part.

Instagram publishing uses **three stages**.

1. Create media
2. Check media status
3. Publish media

13. Instagram Step 1 — create media container

You successfully used:

```
POST
https://graph.instagram.com/v25.0/YOUR_IG_USER_ID/media
```

Postman

Go to:

```
Body
→ x-www-form-urlencoded
```

Add:

```
image_url = YOUR_PUBLIC_IMAGE_URL
caption = My first Instagram API test
access_token = YOUR_INSTAGRAM_ACCESS_TOKEN
```

Example:

```
POST
https://graph.instagram.com/v25.0/YOUR_IG_USER_ID/media
```

Body:

```
image_url
https://images.unsplash.com/...

caption
My first Instagram API test

access_token
YOUR_TOKEN
```

Successful response:

```
{
  "id": "CONTAINER_ID"
}
```

You got:

```
18101282894129538
```

That is a **container ID**.

It is not the final post ID.

14. Instagram Step 2 — check status

Next:

```
GET
https://graph.instagram.com/v25.0/CONTAINER_ID
```

Use query parameters:

```
fields=status_code,status
access_token=YOUR_TOKEN
```

Example:

```
GET
https://graph.instagram.com/v25.0/18101282894129538
```

Params:

```
fields = status_code,status
access_token = YOUR_TOKEN
```

Successful response:

```
{
  "status_code": "FINISHED",
  "status": "FINISHED",
  "id": "18101282894129538"
}
```

You successfully reached this stage.

15. Instagram Step 3 — publish

Then:

```
POST
https://graph.instagram.com/v25.0/YOUR_IG_USER_ID/media_publish
```

Body:

```
creation_id = CONTAINER_ID
access_token = YOUR_TOKEN
```

Example:

```
creation_id
18101282894129538

access_token
YOUR_TOKEN
```

Response:

```
{  
  "id": "PUBLISHED_MEDIA_ID"  
}
```

And you verified the image actually appeared on Instagram.

So this part is 100% proven.

16. INSTAGRAM FINAL UNDERSTANDING

You should remember this pattern:

```
IG_USER_ID  
  |  
  ▼  
/media  
  |  
  ▼  
CONTAINER_ID  
  |  
  ▼  
check status  
  |  
  ▼  
FINISHED  
  |  
  ▼  
/media_publish  
  |  
  ▼  
Instagram post
```

This exact pattern will later become your n8n workflow.

17. FACEBOOK — COMPLETE SETUP

Now Facebook.

Your Facebook architecture is different.

Instagram:

Instagram Account



Instagram Access Token

Facebook:

Facebook User



Facebook Page



Page Access Token

This distinction is extremely important.

18. Facebook Page

You created a Facebook Page:

my social publisher

You already verified the Page ID through Meta.

Your Page ID should be stored as:

YOUR_PAGE_ID

Do not hard-code it into frontend JavaScript.

19. Facebook use case

Inside your Meta app:

Use cases

you selected:

Manage everything on your Page

Then:

```
Customize  
→ Permissions and features
```

You configured the important Page permissions.

You verified:

```
pages_manage_posts  
pages_read_engagement  
pages_show_list
```

and Meta showed them:

```
Ready for testing
```

20. Facebook Login for Business

Open:

```
Facebook Login for Business
```

Then:

```
Configurations
```

Create a configuration.

You created:

```
Facebook Page Publishing
```

21. Configuration access token type

You selected:

User access token

This was the correct choice for the initial user-authorized setup.

The configuration contained Page permissions.

Meta created the configuration successfully.

22. Facebook Testing

Go to:

Testing

Under:

Manage everything on your Page

you saw the permission test requirements.

Then you clicked:

Open Graph API Explorer

23. Facebook User Access Token

In Graph API Explorer:

Meta App
→ VelorQ
→ User Token
→ Generate Access Token

Meta generates a **User Access Token**.

Important:

This is **not yet the final Page access token** you should use for Page publishing.

You use it to ask Meta:

Which Pages does this Facebook user manage?

24. Get the Facebook Page

In Graph API Explorer you successfully called:

```
GET
https://graph.facebook.com/v26.0/me/accounts?fields=id,name,access_token,tasks
```

This returned something like:

```
{
  "data": [
    {
      "id": "YOUR_PAGE_ID",
      "name": "my social publisher",
      "access_token": "PAGE_ACCESS_TOKEN",
      "tasks": [
        "MODERATE",
        "MESSAGING",
        "ANALYZE",
        "ADVERTISE",
        "CREATE_CONTENT",
        "MANAGE"
      ]
    }
  ]
}
```

This is one of the most important concepts in Facebook API development.

25. User Token vs Page Token

Think:

```
USER ACCESS TOKEN
  ↓
/me/accounts
  ↓
PAGE ACCESS TOKEN
```

The Page access token is the token you use for Page operations.

So:

```
Facebook User Token
```

is not the same thing as:

```
Facebook Page Token
```

For your Page API calls, use:

```
PAGE_ACCESS_TOKEN
```

26. Your Facebook test

You then tested:

```
POST
https://graph.facebook.com/v26.0/YOUR_PAGE_ID/feed
```

Body:

```
message = Facebook API test
access_token = YOUR_PAGE_ACCESS_TOKEN
```

This worked.

You got:

```
{
  "id": "...",
  "post_id": "..."
}
```

So:

Facebook text publishing is working.

27. Important correction: Facebook IMAGE test

Your original goal is not just text.

You want:

```
Image
+
Caption
```

For that, we still need to complete one Facebook photo test.

The intended endpoint is:

```
POST
https://graph.facebook.com/v26.0/YOUR_PAGE_ID/photos
```

Body:

```
url = PUBLIC_IMAGE_URL
caption = My first Facebook image test
access_token = YOUR_PAGE_ACCESS_TOKEN
```

Postman:

```
Authorization
→ No Auth
```

```
Body
→ x-www-form-urlencoded
```

Then:

```
url
https://images.unsplash.com/...

caption
My first Facebook image test

access_token
YOUR_PAGE_ACCESS_TOKEN
```

You previously received:

```
(#200) publish_actions has been deprecated
```

That was during your earlier test. The likely reason was that the request was being evaluated with the wrong token/authentication context rather than the Page token we later extracted.

So I would test the photo endpoint again **using the Page access token returned from `/me/accounts` and no separate Authorization header.**

Do not call Facebook image publishing fully complete until you actually see the photo on the Page.

28. WHERE THE TOKENS LIVE

This is another major concept.

Instagram

Instagram Account A

↓

Instagram Token A

Instagram Account B

↓

Instagram Token B

Facebook

Facebook Page A

↓

Page Token A

Facebook Page B

↓

Page Token B

So for multiple customers your database might eventually contain:

```
users
```

```
-----
```

```
user_id
```

```
name

social_accounts
-----
id
user_id
platform
account_id
account_name
access_token
expires_at
```

Example:

```
user 1
Instagram → account A → token A

user 1
Facebook → page A → token B

user 2
Instagram → account B → token C

user 2
Facebook → page B → token D
```

That is how your future multi-account system works.

29. NEVER put tokens in the frontend

Bad:

```
React / HTML / JavaScript
    ↓
Instagram Token
Facebook Token
```

Good:

```
Frontend
    ↓
Backend
    ↓
n8n
```

↓
Meta

The frontend sends:

```
{
  "platform": "instagram",
  "caption": "Hello",
  "image_url": "https://storage.example.com/a.jpg"
}
```

It does **not** send:

```
{
  "access_token": "..."
}
```

Your backend/n8n finds the correct token securely.

30. NOW n8n — STARTING FROM ZERO

Now we get to the part your original project is actually about.

Your first n8n workflow should only support Instagram.

Don't try to build Instagram + Facebook + Both immediately.

Build:

```
Webhook
↓
Instagram Create Media
↓
Check Status
↓
Publish
```

Once this works, clone/extend it for Facebook.

31. n8n — create workflow

Open your n8n.

Create:

New Workflow

Call it:

Social Media Publisher

32. Node 1 — Webhook

Add:

Webhook

Set:

HTTP Method:
POST

Set path:

social/publish

Your test URL will look roughly like:

`https://YOUR-N8N-DOMAIN/webhook/social/publish`

If you're running n8n locally, it may instead be something like:

`http://localhost:5678/webhook-test/social/publish`

The exact hostname depends on your n8n setup.

33. Data sent into n8n

For the first Instagram test, send:

```
{  
  "platform": "instagram",  
  "image_url": "https://images.unsplash.com/...",  
  "caption": "Hello from n8n"  
}
```

This becomes the input to your workflow.

34. Node 2 — Edit Fields

Add:

Edit Fields

Purpose:

Clean and normalize the incoming data.

For example:

```
platform  
image_url  
caption
```

You can later add:

```
user_id  
social_account_id
```

For now keep it simple.

35. Node 3 — Switch

Add:

Switch

Set it to inspect:

platform

Possible values:

instagram
facebook
both

Your routing becomes:

```
graph LR
    Webhook --> Switch
    Switch --> instagram
    Switch --> facebook
    Switch --> both
```

But initially, only build:

instagram

36. Instagram Node 1 — Create Media

Add:

HTTP Request

Name it:

Instagram - Create Media

Method

POST

URL

```
https://graph.instagram.com/v25.0/YOUR_IG_USER_ID/media
```

Do not hard-code your actual user ID in your final production design if you will support many users.

For your first test it is fine to use a fixed test account.

37. Instagram body

Choose:

```
Body  
→ Form-URLencoded
```

Add:

```
image_url
```

Value:

```
{{$.json.image_url}}
```

Then:

```
caption
```

Value:

```
{{$.json.caption}}
```

Then:

```
access_token
```

Value:

```
YOUR_INSTAGRAM_TOKEN
```

For your first learning version, you can place the test token there.

But before production, move the token into n8n's credential/secret storage.

38. Response from Create Media

You should get:

```
{
  "id": "1810..."
}
```

n8n now has:

```
$json.id
```

That is your container ID.

39. Instagram Node 2 — Check Status

Add another:

```
HTTP Request
```

Name:

```
Instagram - Check Status
```

Method:

```
GET
```

URL:

```
https://graph.instagram.com/v25.0/{{ $json.id }}
```

Query Parameters:

```
fields = status_code,status
access_token = YOUR_INSTAGRAM_TOKEN
```

40. Result

You want:

```
{
  "status_code": "FINISHED",
  "status": "FINISHED"
}
```

41. IF node

Add:

IF

Condition:

```
{{$.json.status_code}}
```

equals:

FINISHED

If yes:

Publish

If no:

Wait

42. Why Wait matters

Sometimes Meta hasn't finished processing the image immediately.

So:

```
Create Media
  ↓
Check Status
  ↓
FINISHED?
├─ YES → Publish
└─ NO  → Wait → Check again
```

This is a very important automation concept.

43. Wait node

Add:

```
Wait
```

For example:

```
5 seconds
```

Then connect:

```
Wait
  ↓
Check Status
```

Now it automatically retries.

44. Instagram Publish Node

Add:

HTTP Request

Name:

Instagram - Publish

Method:

POST

URL:

`https://graph.instagram.com/v25.0/YOUR_IG_USER_ID/media_publish`

Body:

```
creation_id = {{$node["Instagram - Create Media"].json["id"]}}  
  
access_token = YOUR_INSTAGRAM_TOKEN
```

Depending on your n8n version, the expression can also reference the current JSON directly if you've preserved the container ID.

45. Instagram result

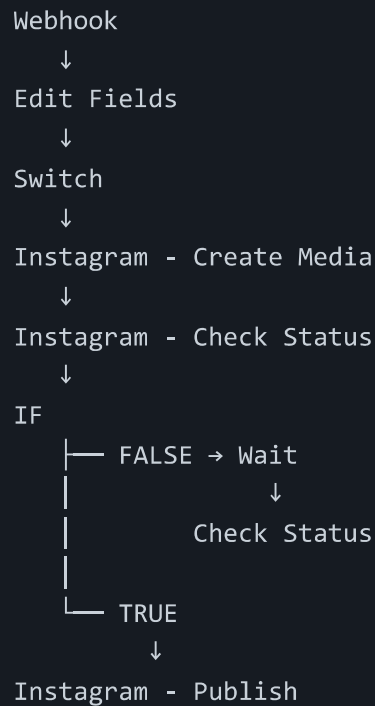
You should receive:

```
{  
  "id": "1789..."  
}
```

That is your published Instagram media ID.

46. FIRST COMPLETE N8N FLOW

Your first successful workflow should be:



That is your first real social-media automation.

47. THEN FACEBOOK

Once Instagram works in n8n, add:

Facebook - Publish

For text testing:

POST
https://graph.facebook.com/v26.0/YOUR_PAGE_ID/feed

Body:

```
message = {{$json.caption}}  
access_token = YOUR_PAGE_TOKEN
```

48. FACEBOOK PHOTO NODE

For image publishing:

POST

https://graph.facebook.com/v26.0/YOUR_PAGE_ID/photos

Body:

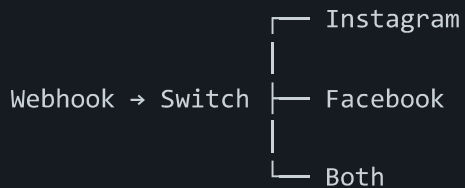
```
url = {{$json.image_url}}
caption = {{$json.caption}}
access_token = YOUR_PAGE_TOKEN
```

Then verify the image appears on the Page.

This is the part we should finish testing in Postman before putting it into n8n.

49. PLATFORM SWITCH

Then your Switch becomes:



50. Instagram branch

```
Instagram
  ↓
Create Media
  ↓
Check Status
  ↓
Publish
```

51. Facebook branch

```
Facebook
  ↓
```

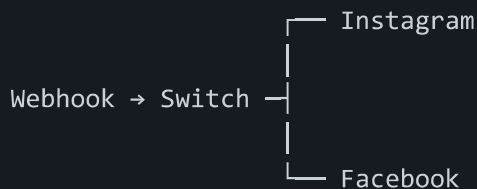
Unlike Instagram, Facebook doesn't require the same container/status/publish sequence for the simple Page photo operation you're testing.

52. BOTH branch

For:

```
platform = both
```

you can run both branches:



Then optionally:



53. What your frontend eventually sends

Your frontend form can be:

```
Upload Image  
[ Choose File ]  
  
Caption  
[_____]
```

Publish To
() Instagram
() Facebook
() Both

[SEND NOW]

When the user clicks Send:

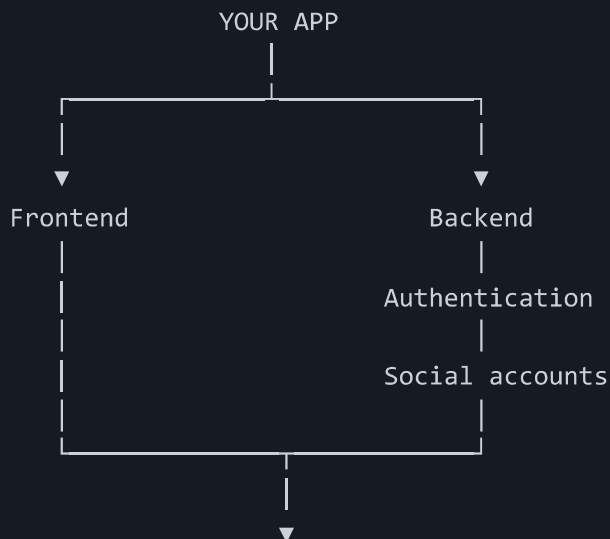
```
Frontend
↓
Upload image
↓
Storage
↓
Get HTTPS image URL
↓
POST to n8n
```

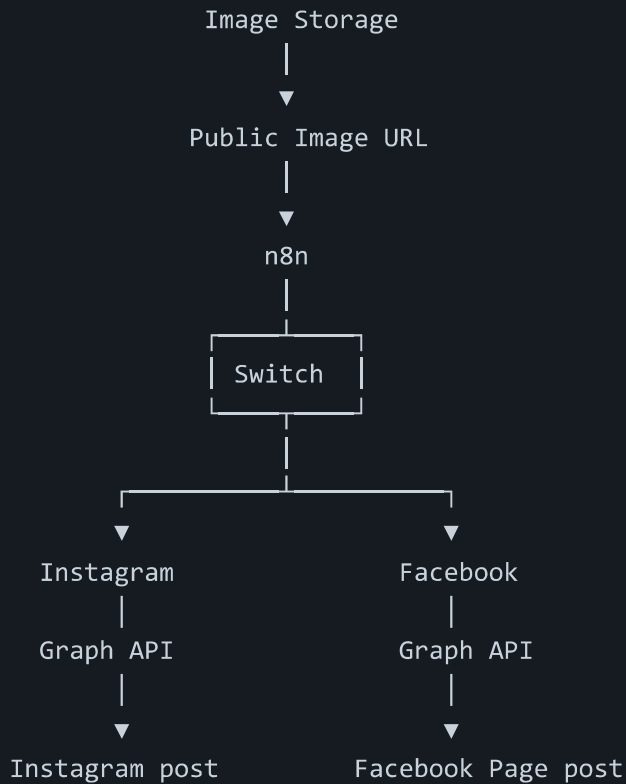
Request:

```
{
  "platform": "both",
  "caption": "New product launch!",
  "image_url": "https://storage.example.com/abc123.jpg"
}
```

54. Your final architecture

Eventually:





55. Token architecture for multiple users

Suppose your application has three customers.

Customer A

- └─ Instagram token A1
- └─ Facebook Page token A2

Customer B

- └─ Instagram token B1
- └─ Facebook Page token B2

Customer C

- └─ Instagram token C1
- └─ Facebook Page token C2

When Customer B presses:

Send Now

your system must load:

Customer B's Instagram token

not Customer A's.

That's why the database must associate tokens with users/accounts.

56. Your database later

Something like:

```
users
-----
id
email
password_hash

social_accounts
-----
id
user_id
platform
account_id
account_name
access_token
token_expires_at
status
```

Example:

```
1 | user_101 | instagram | ig_111 | @brand1 | TOKEN...
2 | user_101 | facebook  | pg_222 | Brand Page | TOKEN...
3 | user_102 | instagram | ig_333 | @brand2 | TOKEN...
```

57. Do not store this in frontend

Never:

```
const facebookToken = "EAAB....";
```

Never:

```
const instagramToken = "IGQ....";
```

Never:

```
.env
```

inside a public frontend project.

Tokens should stay server-side.

58. YOUR TESTING CHECKLIST

Instagram

You have:

- ☒ Professional account
- ☒ Meta app
- ☒ Instagram use case
- ☒ Required permissions
- ☒ Instagram Tester
- ☒ Tester invitation accepted
- ☒ Instagram account added
- ☒ Instagram token
- ☒ Token validation
- ☒ Instagram account ID
- ☒ Create media
- ☒ Check status
- ☒ Publish image
- ☒ Actual Instagram post

Instagram = complete for the test account.

Facebook

You have:

- ☒ Facebook Page
- ☒ Facebook use case
- ☒ Page permissions
- ☒ Facebook Login for Business
- ☒ Configuration
- ☒ User access token
- ☒ /me/accounts

- ☒ Page ID
- ☒ Page access token
- ☒ Facebook text post
- ☐ Facebook image post

So your only missing API proof is:

Facebook Page image publishing

59. Postman test order you should remember

Instagram

```
POST
/IG_USER_ID/media

↓

GET
/CONTAINER_ID?fields=status_code,status

↓

POST
/IG_USER_ID/media_publish
```

Facebook

Text:

```
POST
/PAGE_ID/feed
```

Image:

```
POST
/PAGE_ID/photos
```

60. URL cheat sheet

For your current tested API versions:

Instagram

Create media:

```
https://graph.instagram.com/v25.0/{IG_USER_ID}/media
```

Check status:

```
https://graph.instagram.com/v25.0/{CONTAINER_ID}
```

Publish:

```
https://graph.instagram.com/v25.0/{IG_USER_ID}/media_publish
```

Facebook

Get Pages:

```
https://graph.facebook.com/v26.0/me/accounts
```

Text Page post:

```
https://graph.facebook.com/v26.0/{PAGE_ID}/feed
```

Photo Page post:

```
https://graph.facebook.com/v26.0/{PAGE_ID}/photos
```

Use your own stored IDs and tokens in place of the `{...}` values.

61. The most important beginner mental model

Whenever you get confused, return to this:

Instagram

WHO?

IG User ID

AUTH?

Instagram Access Token

WHAT?

Image URL + Caption

HOW?

Create → Check → Publish

Facebook

WHO?

Page ID

AUTH?

Page Access Token

WHAT?

Image + Caption

HOW?

Page endpoint

62. What we should do next

Don't start the frontend yet.

The clean order is:

1. Finish Facebook image Postman test
↓
2. Build Instagram n8n workflow
↓
3. Build Facebook n8n workflow
↓
4. Build Both switch
↓
5. Store tokens securely in n8n/backend
↓
6. Build image storage
↓
7. Build frontend
↓

8. Connect frontend → backend → n8n

↓

9. Multi-user OAuth

That order will make debugging much easier because every layer is proven separately.

The **very next technical task** should be the Facebook image test, using the **Page access token** from `/me/accounts` , followed by the Instagram-only n8n workflow using the exact three requests you already proved in Postman.